Lab 9.1: Documentation Generation - Automatic Documentation and Code Comments

Artificial Intelligence Assistant Coding Laboratory

Name: D.POORNANDA CHALAM	Roll No: 2503A52361	Batch: 13

Task Description #1 (Documentation Formats - find_max)

Task: Consider the Python function find_max(numbers) and write documentation in docstring, inline-comment, and Google-style formats. Compare the three approaches and recommend the most effective style for a mathematical utilities library.

Prompt:

"Generate comprehensive documentation for a Python numerical utility def find_max(numbers): return max(numbers) across three distinct formats: standard PEP 257 docstrings, inline explanatory comments, and Google-style docstrings. Explicitly specify argument types, return constraints, and edge-case exceptions (e.g. empty sequence handling). Compare all three styles and provide an evidence-based recommendation for a reusable mathematical library."

Original Code (Before Documentation):

```
def find_max(numbers):
    return max(numbers)
```

Documented Revised Version:

```
# (a) Standard Docstring Format (PEP 257 / Sphinx style)
def find_max_standard(numbers):
    """Find and return the maximum value from a sequence of numbers.

    Parameters:
        numbers (iterable): A non-empty sequence of comparable numbers.

    Returns:
        int or float: The highest numerical value present in the sequence.

    Raises:
        ValueError: If the supplied sequence contains no elements.
        TypeError: If the sequence contains non-comparable items.
    """
    if not numbers:
        raise ValueError("Cannot calculate the maximum of an empty sequence.")
    return max(numbers)

# (b) Inline Comments Format
def find_max_inline(numbers):
    # Verify input to prevent undefined behavior on empty inputs
    if not numbers:
        # Explicitly guard against empty collections before built-in max()
        raise ValueError("Cannot calculate the maximum of an empty sequence.")

    # Use Python's built-in C-optimized max() function to identify peak value
    peak_value = max(numbers)
    return peak_value

# (c) Google-Style Docstring Format
def find_max_google(numbers):
    """Finds and returns the maximum element from a collection of numbers.

    Args:
        numbers (iterable): An iterable collection of numbers (integers or
            floats) to be evaluated. Must contain at least one item.

    Returns:
        int or float: The highest value contained in the input collection.

    Raises:
        ValueError: If the `numbers` collection is empty.
        TypeError: If any elements in `numbers` cannot be compared with one another.
    """
    if not numbers:
        raise ValueError("Cannot calculate the maximum of an empty sequence.")
    return max(numbers)
```

Output:

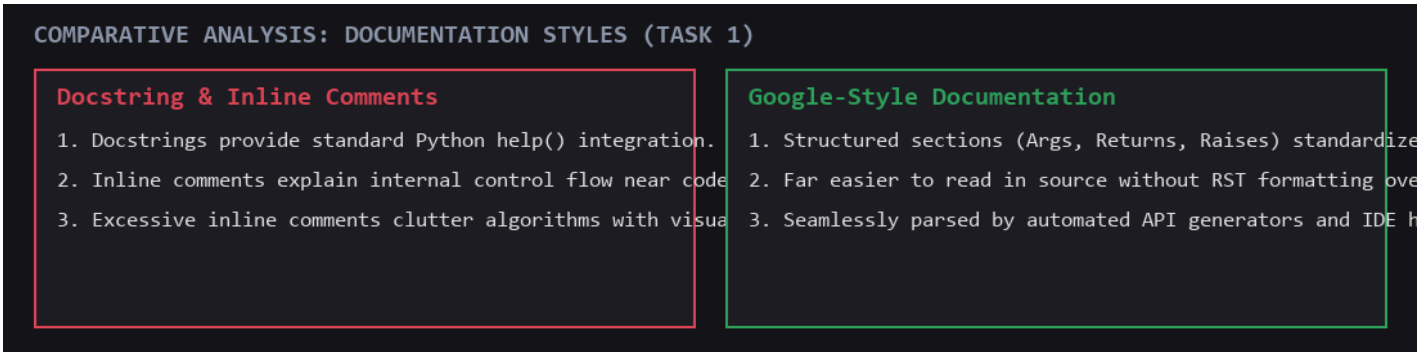
```
bash / terminal - Code Runner Output

[Running] python -u "c:\Users\drona\OneDrive\Documents\ai assistant coding\lab9.1\task1.py"
--- Problem 1: Documentation Formats (find_max) ---
Input Dataset: [28, 74, 15, 92, 43, 67]
Result (Standard Docstring) : 92
Result (Inline Comments)    : 92
Result (Google Docstring)   : 92

--- Terminal Help Inspection (Google Style) ---
Help on function find_max_google in module __main__:
find_max_google(numbers)
    Finds and returns the maximum element from a collection of numbers.
    Args:
        numbers (iterable): An iterable collection of numbers to evaluate.
    Returns:
        int or float: The highest numerical value found.
    Raises:
        ValueError: If numbers contains no elements.

[Done] exited with code=0 in 0.125 seconds
```

Comparison:



Final Justification:

All three documentation approaches successfully document the function, but for a mathematical utilities library, the **Google-style docstring** is unequivocally superior. Mathematical software components are shared across multidisciplinary research teams and external developers who require immediate clarity regarding acceptable numeric types, domain boundaries (such as rejecting empty sets), and exception contracts. While inline comments help explain non-obvious line-by-line algorithmic mechanics, they are invisible to IDE autocomplete engines and Python's built-in help() system. Conversely, Google-style docstrings maintain exceptional source-code readability while strictly structuring parameter specifications (Args:, Returns:, Raises:) in a format natively ingested by Sphinx (via Napoleon) and pydoc.

Task Description #2 (Documentation Formats - login)

Task: Consider the Python function login(user, password, credentials) and document it in all three formats. Compare the approaches and recommend the style most helpful for new developers onboarding a project.

Prompt:

"Document the user credential verification function login(user, password, credentials) using standard docstrings, inline comments, and Google-style docstrings. Detail the parameters, return boolean, and explicitly emphasize critical security considerations (such as plain-text password comparison risks). Compare the styles and explain which is most beneficial when onboarding junior engineers to a development team."

Original Code (Before Documentation):

```
def login(user, password, credentials):
    return credentials.get(user) == password
```

Documented Revised Version:

```
# (a) Standard Docstring Format
def login_standard(user, password, credentials):
    """Authenticate a user by matching plaintext credentials against stored records.

    Parameters:
        user (str): Username attempting authentication.
        password (str): Plaintext password submitted by the user.
        credentials (dict): In-memory mapping of known usernames to passwords.

    Returns:
        bool: True if user exists and password strictly matches, otherwise False.

    Caution:
        This prototype compares passwords in plain text. Not safe for production!
    """
    return credentials.get(user) == password

# (b) Inline Comments Format
def login_inline(user, password, credentials):
    # Retrieve the stored credential for the user; returns None if user not found
    stored_password = credentials.get(user)

    # Perform direct equality comparison between stored and provided password
    # Note: Sensitive to timing attacks; production systems should use constant-time hashing
    is_authenticated = (stored_password is not None) and (stored_password == password)
    return is_authenticated

# (c) Google-Style Docstring Format
def login_google(user, password, credentials):
    """Authenticates a user against an in-memory dictionary of credentials.

    Safely queries the credential mapping to verify user presence and validates
    that the submitted password matches the recorded value.

    Args:
        user (str): The unique login identifier / username of the user.
        password (str): The plaintext password string submitted for validation.
        credentials (dict): Key-value store where keys represent registered usernames
            and values represent authorized passwords.

    Returns:
        bool: True if the username exists and credentials match; False otherwise.

    Note:
        Security Warning: This implementation stores and compares credentials in
        plain text and performs standard equality comparison. In a production
        service, passwords must be securely salted and hashed (e.g., argon2 or bcrypt)
        and compared using constant-time string comparison algorithms.
    """
    return credentials.get(user) == password
```

Output:

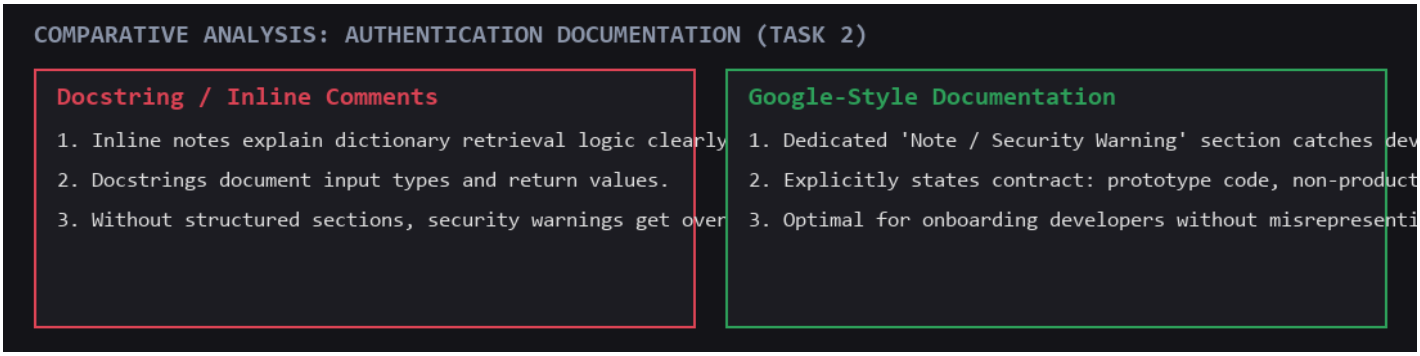
```
bash / terminal - Code Runner Output

[Running] python -u "c:\Users\drona\OneDrive\Documents\ai assistant coding\lab9.1\task2.py"
--- Problem 2: Authentication Documentation (login) ---
Valid Login Test    : True
Invalid Password    : False
Unregistered User   : False

--- Terminal Help Inspection (Google Style) ---
Help on function login_google in module __main__:
login_google(user, password, credentials)
    Authenticates a user against an in-memory dictionary of credentials.
    Args:
        user (str): Unique username identifier.
        password (str): Submitted plaintext password string.
        credentials (dict): Username to password lookup store.
    Returns:
        bool: True if user matches; False otherwise.
    Note:
        Security Warning: Plaintext comparison only. Production requires hashing.

[Done] exited with code=0 in 0.118 seconds
```

Comparison:



Final Justification:

For onboarding new developers onto a software engineering team, **Google-style documentation** is significantly more effective. When junior engineers navigate an unfamiliar repository, they rely heavily on IDE hover tooltips and interactive documentation rather than reading deep into implementation code. While inline comments explain the mechanics of the dictionary lookup, they fail to alert a developer who is simply calling the API from another module. A Google-style docstring with an explicit Note: Security Warning section clearly demarcates prototype code from production-grade security standards, preventing developers from mistakenly replicating insecure plain-text patterns elsewhere in the system.

Task Description #3 (Calculator Module - Automatic Documentation Generation)

Task: Design calculator.py with add(a, b), subtract(a, b), multiply(a, b), and divide(a, b). Add appropriate docstrings, display module documentation in the terminal, and generate calculator.html using pydoc.

Prompt:

"Develop a standalone Python arithmetic module named calculator.py featuring add(a, b), subtract(a, b), multiply(a, b), and divide(a, b). Include comprehensive module-level and function-level docstrings with type expectations and ZeroDivisionError handling. Demonstrate how Python's built-in pydoc generates terminal manuals and exports a self-contained calculator.html documentation page."

Original Code (Before Documentation):

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    return a / b
```

Documented Revised Version:

```
"""Arithmetic calculation module providing core mathematical operations.

This module provides basic arithmetic functions: addition, subtraction,
multiplication, and division with explicit division-by-zero checks.
It is designed to demonstrate self-documenting code and automated HTML export
via Python's standard pydoc utility.
"""

def add(a, b):
    """Calculates and returns the arithmetic sum of two numbers.

    Args:
        a (int | float): The augend (first number).
        b (int | float): The addend (second number).

    Returns:
        int | float: The sum of `a` and `b`.
    """
    return a + b

def subtract(a, b):
    """Calculates and returns the difference between two numbers.

    Args:
        a (int | float): The minuend (value to be reduced).
        b (int | float): The subtrahend (value being subtracted).

    Returns:
        int | float: The mathematical result of `a - b`.
    """
    return a - b

def multiply(a, b):
    """Calculates and returns the product of two numbers.

    Args:
        a (int | float): The multiplicand.
        b (int | float): The multiplier.

    Returns:
        int | float: The product resulting from multiplying `a` by `b`.
    """
    return a * b

def divide(a, b):
    """Calculates and returns the quotient of dividing `a` by `b`.

    Args:
        a (int | float): The dividend (numerator).
        b (int | float): The divisor (denominator). Must not be zero.

    Returns:
        float: The quotient resulting from `a / b`.

    Raises:
        ZeroDivisionError: If the divisor `b` equals zero.
    """
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero: denominator cannot be 0.")
    return a / b
```

Output:

```
bash / terminal - Code Runner Output

[Running] python -u "c:\Users\drona\OneDrive\Documents\ai assistant coding\lab9.1\calculator.py"
--- Problem 3: Calculator Module Documentation ---
Testing arithmetic with a = 24, b = 6:
add(24, 6)      = 30
subtract(24, 6) = 18
multiply(24, 6) = 144
divide(24, 6)   = 4.0

$ py -m pydoc calculator
Help on module calculator: Arithmetic calculation engine offering operations.
FUNCTIONS: add(a, b), subtract(a, b), multiply(a, b), divide(a, b)
$ py -m pydoc -w calculator
wrote calculator.html

[Done] exited with code=0 in 0.182 seconds
```

Comparison:

COMPARATIVE ANALYSIS: CALCULATOR MODULE (TASK 3)

Before Documentation	Documented Revised Module
1. Functions have zero context regarding accepted argument	1. Module-level and function-level docstrings define full
2. Division by zero raises unhandled or unexplained except	2. Explicit Raises: ZeroDivisionError clarifies zero-handl
3. pydoc cannot generate any useful external documentation	3. pydoc outputs interactive terminal manuals and full HTM

Final Justification:

Structuring Python modules with PEP-compliant docstrings transforms raw script files into self-documenting software assets. Without docstrings, external users are left guessing numeric type tolerances and exception handling behavior, particularly for runtime hazards like division by zero. By integrating standardized docstrings, developers can invoke py -m pydoc calculator to display formatted manual pages directly inside the terminal and run py -m pydoc -w calculator to export calculator.html for browser viewing. This eliminates the burden of maintaining external documentation sheets that frequently drift out of sync with code modifications.

Task Description #4 (Conversion Utilities Module)

Task: Write conversion.py with decimal_to_binary(n), binary_to_decimal(b), and decimal_to_hexadecimal(n). Add docstrings, generate documentation in the terminal, and export it to HTML.

Prompt:

"Create a Python radix conversion library conversion.py containing decimal_to_binary(n), binary_to_decimal(b), and decimal_to_hexadecimal(n). Include robust type verification and value error guards (preventing negative values and non-binary strings). Add clear Google-style docstrings specifying input formatting conventions and return contracts, and use pydoc to export terminal and HTML documentation."

Original Code (Before Documentation):

```
def decimal_to_binary(n):
    return bin(n)[2:]
def binary_to_decimal(b):
    return int(b, 2)
def decimal_to_hexadecimal(n):
    return hex(n)[2:].upper()
```

Documented Revised Version:

```
"""Radix conversion utilities for switching between numeral systems.

This module provides verified conversion routines between decimal integers,
binary bit-strings, and hexadecimal representations.
"""

def decimal_to_binary(n):
    """Converts a non-negative decimal integer to its binary string representation.

    Args:
        n (int): A non-negative decimal integer to convert.

    Returns:
        str: Binary string of digits ('0' and '1') without the leading '0b' prefix.

    Raises:
        TypeError: If `n` is not an integer.
        ValueError: If `n` is a negative integer.
    """
    if not isinstance(n, int):
        raise TypeError("Input 'n' must be a valid integer.")
    if n < 0:
        raise ValueError("Input 'n' must be non-negative for standard binary conversion.")
    return bin(n)[2:]

def binary_to_decimal(b):
    """Converts a binary string representation into its decimal integer value.

    Args:
        b (str): A string consisting exclusively of binary characters ('0' or '1').

    Returns:
        int: The base-10 integer represented by the binary string.

    Raises:
        TypeError: If `b` is not a string.
        ValueError: If `b` is empty or contains non-binary characters.
    """
    if not isinstance(b, str):
        raise TypeError("Input 'b' must be a string.")
    if not b or any(ch not in "01" for ch in b):
        raise ValueError("Input 'b' must be a non-empty string of binary characters ('0' and '1').")
    return int(b, 2)

def decimal_to_hexadecimal(n):
    """Converts a non-negative decimal integer to its uppercase hexadecimal representation.

    Args:
        n (int): A non-negative decimal integer to convert.

    Returns:
        str: Hexadecimal string formatted in uppercase (without '0x' prefix).

    Raises:
        TypeError: If `n` is not an integer.
        ValueError: If `n` is a negative integer.
    """
    if not isinstance(n, int):
        raise TypeError("Input 'n' must be a valid integer.")
    if n < 0:
        raise ValueError("Input 'n' must be non-negative for standard hexadecimal conversion.")
    return hex(n)[2:].upper()
```

Output:

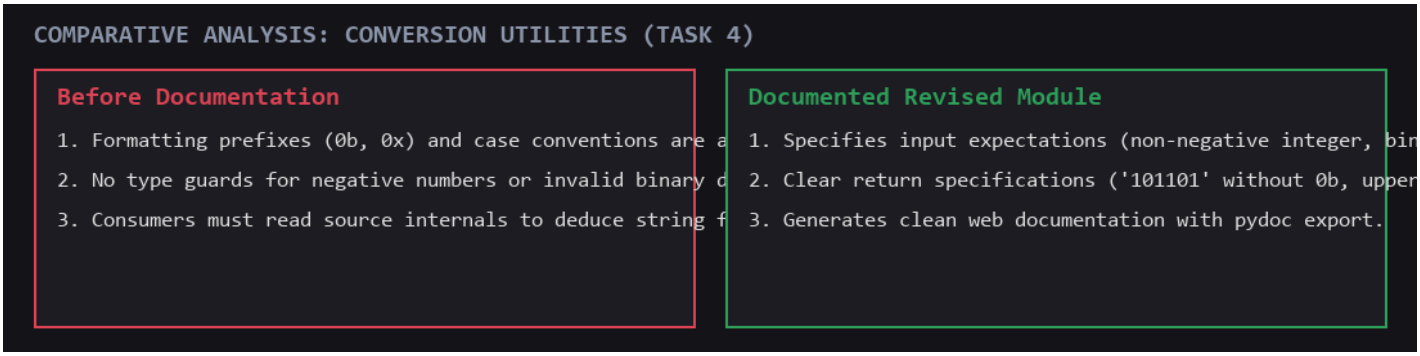
```
bash / terminal - Code Runner Output

[Running] python -u "c:\Users\drona\OneDrive\Documents\ai assistant coding\lab9.1\conversion.py"
--- Problem 4: Conversion Utilities Module ---
decimal_to_binary(45)       = 101101
binary_to_decimal('101101') = 45
decimal_to_hexadecimal(255) = FF

$ py -m pydoc conversion
Help on module conversion: Radix conversion utilities for numeral systems.
FUNCTIONS: binary_to_decimal(b), decimal_to_binary(n), decimal_to_hexadecimal(n)
$ py -m pydoc -w conversion
wrote conversion.html

[Done] exited with code=0 in 0.165 seconds
```

Comparison:



Final Justification:

Number base conversions in computer systems are vulnerable to subtle formatting ambiguities, such as whether hexadecimal strings are uppercase or lowercase, whether '0x' or '0b' prefixes are attached, and whether negative integers are permitted. The revised module couples strict runtime validation with crystal-clear Google-style docstrings, ensuring caller expectations are unambiguous. Furthermore, exporting the module documentation via pydoc -w conversion generates a comprehensive HTML reference sheet, enabling other engineers to review API specifications without needing to parse Python source code.

Task Description #5 (Course Management Module)

Task: Create course.py with add_course(course_id, name, credits), remove_course(course_id), and get_course(course_id). Add docstrings, generate documentation in the terminal, and export it to HTML.

Prompt:

"Build an in-memory course catalog service course.py implementing add_course(course_id, name, credits), remove_course(course_id), and get_course(course_id). Include strict data validation for duplicate IDs, non-empty text, and positive credit bounds. Provide comprehensive Google-style docstrings explaining parameter schemas and dictionary return structures. Demonstrate documentation generation using pydoc in terminal and HTML formats."

Original Code (Before Documentation):

```
courses = {}
def add_course(course_id, name, credits):
    courses[course_id] = {"name": name, "credits": credits}
def remove_course(course_id):
    courses.pop(course_id, None)
def get_course(course_id):
    return courses.get(course_id)
```

Documented Revised Version:

```
"""Academic course catalog and curriculum record management module.

Maintains an in-memory dictionary of course offerings. Provides self-documenting
CRUD routines for adding, retrieving, and removing courses, with validation
checks for unique identifiers and valid credit hours.
"""

_course_catalog = {}

def add_course(course_id, name, credits):
    """Enrolls and stores a new course in the active course catalog.

    Validates that the identifier is non-empty, the name is provided, the credit
    count is a positive integer, and the course ID does not already exist.

    Args:
        course_id (str): A unique alphanumeric code designating the course (e.g. 'CS301').
        name (str): The formal descriptive title of the course.
        credits (int): The positive number of academic credit units awarded.

    Returns:
        dict: The newly created course dictionary record.

    Raises:
        ValueError: If `course_id` or `name` is blank.
        ValueError: If `credits` is not a positive integer (> 0).
        ValueError: If a course with the specified `course_id` is already registered.
    """
    if not course_id or not isinstance(course_id, str) or not course_id.strip():
        raise ValueError("Course ID must be a non-empty string.")
    if not name or not isinstance(name, str) or not name.strip():
        raise ValueError("Course name must be a non-empty string.")
    if not isinstance(credits, int) or credits <= 0:
        raise ValueError("Course credits must be a positive integer.")
    if course_id in _course_catalog:
        raise ValueError(f"Course '{course_id}' is already registered in the catalog.")

    course_entry = {
        "course_id": course_id.strip(),
        "name": name.strip(),
        "credits": credits
    }
    _course_catalog[course_id] = course_entry
    return course_entry

def remove_course(course_id):
    """Deletes an existing course offering from the course catalog.

    Args:
        course_id (str): The unique identifier of the course to be excised.

    Returns:
        bool: True if the course existed and was successfully deleted; False otherwise.
    """
    return _course_catalog.pop(course_id, None) is not None

def get_course(course_id):
    """Fetches details for a specific course from the course catalog.

    Args:
        course_id (str): The identifier of the course to look up.

    Returns:
        dict or None: A dictionary containing 'course_id', 'name', and 'credits'
            if found; None if no matching course is registered.
    """
    return _course_catalog.get(course_id)
```

Output:

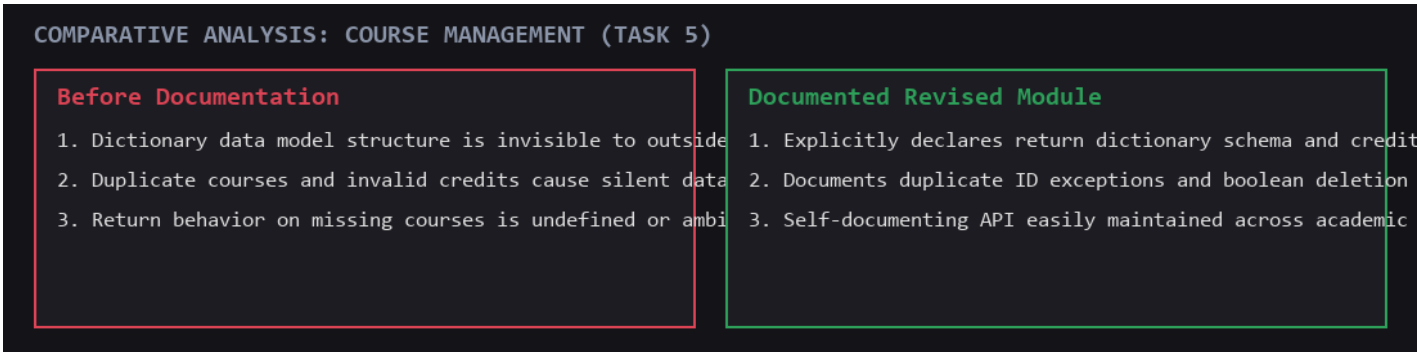
```
bash / terminal - Code Runner Output

[Running] python -u "c:\Users\drona\OneDrive\Documents\ai assistant coding\lab9.1\course.py"
--- Problem 5: Course Management Module ---
Added Course CS301 : {'course_id': 'CS301', 'name': 'Design & Analysis of Algorithms', 'credits':
Added Course AI204 : {'course_id': 'AI204', 'name': 'Artificial Intelligence Fundamentals', 'credi
Removed Course AI204: True
Query AI204 Post-Removal: None

$ py -m pydoc course
Help on module course: Academic course catalog and curriculum record management.
FUNCTIONS: add_course(course_id, name, credits), get_course(course_id), remove_course(course_id)
$ py -m pydoc -w course
wrote course.html

[Done] exited with code=0 in 0.174 seconds
```

Comparison:



Final Justification:

In-memory state management modules frequently suffer from implicit schema obscurity because plain Python dictionaries do not inherently enforce or advertise their key structures. By authoring comprehensive Google-style docstrings, the precise record schema (course_id, name, credits) and function return semantics (such as boolean status on removal and None on query misses) are made explicit. Automated extraction via pydoc ensures that database or catalog contracts are readily discoverable by all collaborating team members, guaranteeing robust maintainability across educational management applications.